EXPERIMENT - 61

1. Problem Statement

Write a C program that reads the size and elements of a variable-length integer array, nums[], containing only positive integers, and an integer target. Pass the array to a function named two_sum() using its address. The function two_sum() must:

- Find two distinct indices, i and j, such that nums[i] + nums[j] = target.
- Display the two indices separated by a space, in any order.
- Display -1 -1 if no such pair exists.
- Use a nested-loop approach, assuming at most one valid solution exists.

2. Algorithm

Main Program

Step 1: START

Step 2: Read n and declare the variable length array nums of size n.

Step 3: Read the n elements of nums, then read target.

Step 4: Call two_sum(&nums[0], n, target).

Step 5: STOP

Function two_sum()

Step 1: Repeat for i = 0 to n − 2:

Repeat for j = i + 1 to n − 1:

If nums[i] + nums[j] is equal to target, display i and j and return.

Step 2: Display −1 −1 and return.

3. Test Case Table

#	Array	Target	Expected Output	Actual Output	Result
1	2 7 11 15	9	0 1, since 2 + 7 = 9	0 1	**PASS**
2	3 2 4	6	1 2, since 2 + 4 = 6	1 2	**PASS**
3	3 3	6	0 1, equal values at different indices	0 1	**PASS**
4	1 2 3	7	−1 −1, no pair adds to 7	−1 −1	**PASS**
5	1 2 3 4 5	9	3 4, the last two elements	3 4	**PASS**
6	5	10	−1 −1, one element cannot pair with itself	−1 −1	**PASS**

#	Array	Target	Expected Output	Actual Output	Result
7	4 1 9 2	6	0 3, the pair is not adjacent	0 3	**PASS**
8	1 1 1 1	2	0 1, the first pair found	0 1	**PASS**

4. Source Code

twosum61.c

```
/* Experiment 61. nums and target are read in main, and the array is
   passed by address, so the function takes int *nums. */

#include <stdio.h>

void two_sum(int *nums, int n, int target);

int main(void)
{
    int n, i, target;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    /* n is known only now, so the array is declared here. */
    int nums[n];

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    printf("Enter the target: ");
    scanf("%d", &target);

    /* &nums[0] is the address the question asks for. */
    two_sum(&nums[0], n, target);

    return 0;
}

/* Nested loops, as the question asks. i picks the first number and j
   tries every number after it.
   j = i + 1 is the part that matters. It keeps the two indices
   distinct, so a single 5 with target 10 gives -1 -1 instead of
   pairing 5 with itself, and it also stops a pair being tried twice.
   return leaves on the first match, so the -1 -1 line only runs when
   both loops finish empty. */
void two_sum(int *nums, int n, int target)
{
    int i, j;

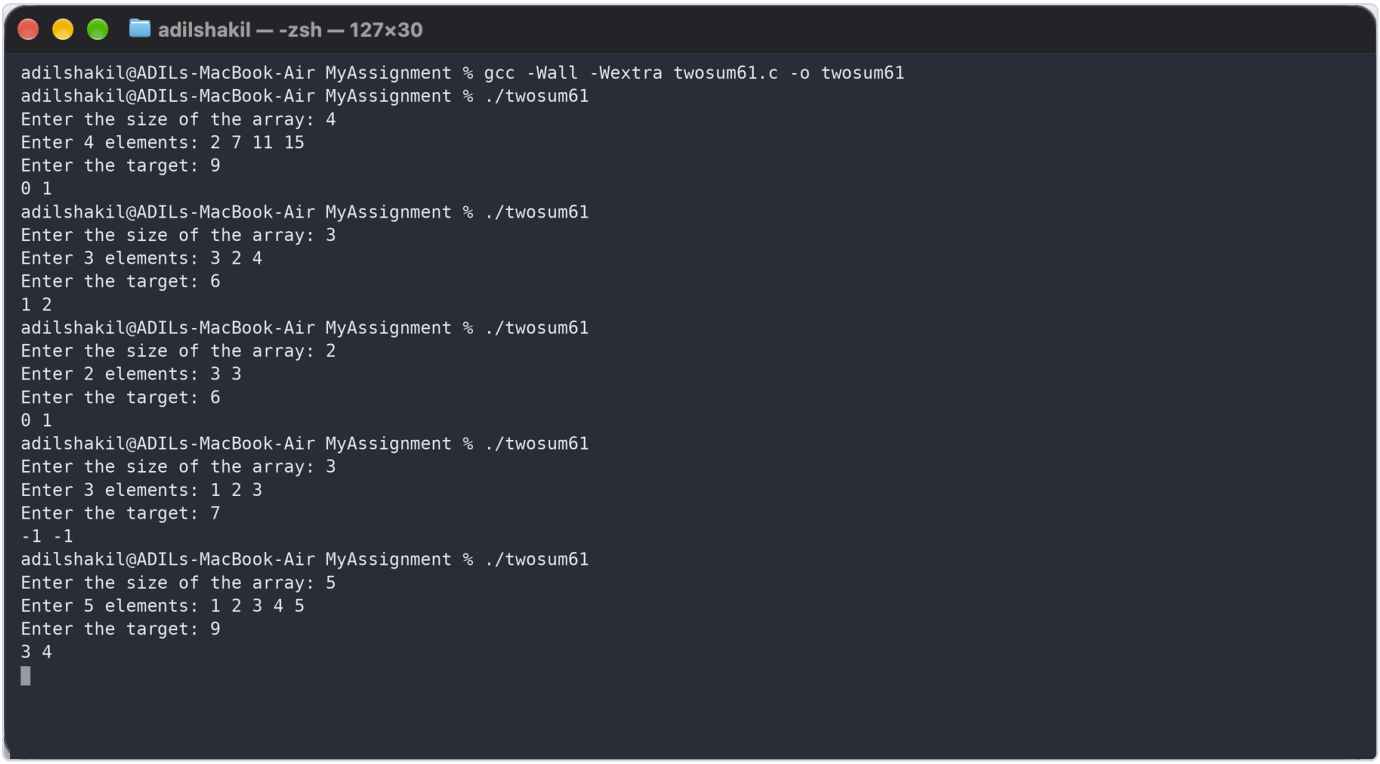
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (nums[i] + nums[j] == target) {
                printf("%d %d\n", i, j);
                return;
            }
        }
    }

    printf("-1 -1\n");
}
```

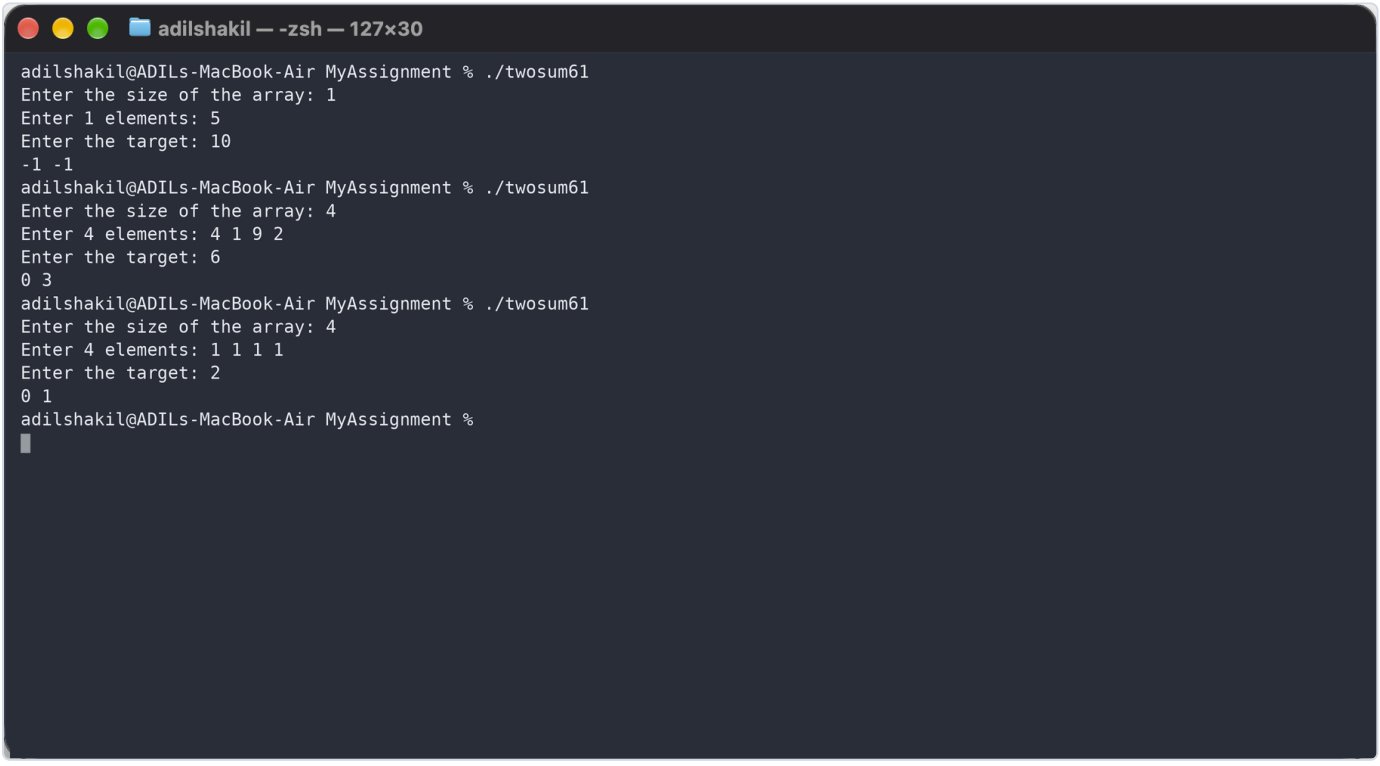
5. Compilation and Output

Compiled with `gcc -Wall -Wextra twosum61.c -o twosum61` so the small warnings would show up too. It built clean.

Cases 1 to 5



Cases 6 to 8



All eight passed, and the first four are the examples from the question.

Case 6 is the one I care about. A single 5 with target 10 must give −1 −1, because 5 + 5 needs the same element twice, and that only comes out right because j starts at i + 1. Case 3 sits next to it: 3 and 3 are the same value but at different indices, so that pair is allowed and the answer is 0 1.